Video Transcript - Part 16

Slide 1

Hey everyone, welcome to part sixteen. In this video we're covering three important deep learning ideas: convolutional neural networks, or CNNs, recurrent neural networks, or RNNs, and a technique that keeps models from memorizing their training data, L1 and L2 regularization.

Slide 2

Here's the plan. We'll start with a quick overview of the three. Then we'll look at how CNNs work and what they're good at, followed by RNNs, and a side by side comparison. After that we'll move to regularization, the difference between L1 and L2, and how you use them in practice.

Slide 3

Here's the quick overview. CNNs are designed for grid like data, mostly images. RNNs are designed for sequences, like text, speech, and time series, where order matters. And regularization is not an architecture at all. It's a technique that works with almost any model to reduce overfitting. So CNNs and RNNs decide what kind of data a network handles well, and regularization helps it generalize.

Slide 4

Let's start with CNNs. A CNN processes an image in stages. First comes the input, the image as a grid of pixel values. Then convolutional layers slide small filters across the image. Each filter looks for a pattern, like an edge or a color change. Early layers find simple things like edges, and deeper layers combine them into shapes and eventually whole objects. Next, pooling layers shrink the data, for example by keeping only the strongest signal in each small region, which makes the network faster and less sensitive to exact positions. Finally, fully connected layers use those features to make a decision, like cat or dog.

Slide 5

So why do CNNs work so well on images? They learn features automatically, so you don't have to design them by hand. They also share parameters, because the same filter is reused across the whole image, which means far fewer weights than a regular network would need. They're used for image classification, object detection, and segmentation, and also in areas like medical imaging and face recognition. In SageMaker, the built in image classification, object detection, and semantic segmentation algorithms are all based on CNNs.

Slide 6
Now RNNs. They're built for sequences, where what came before matters. Think of reading a sentence. The meaning of a word depends on the words earlier. An RNN reads one step at a time, for example one word or one time point. At each step it updates a hidden state, which acts like a short term memory of what it has seen so far. It passes that memory forward to the next step. And it can produce an output at each step, or one final output at the end, like a forecast or a sentiment label.

Slide 7
RNNs shine on text, speech, and time series data. But basic RNNs have a known weakness. Their memory fades over long sequences, because the training signal shrinks as it travels back through many steps, which is called the vanishing gradient problem. The fix is variants called LSTM and GRU, which use gates to decide what to keep and what to forget, so they can remember information over longer spans. In SageMaker, DeepAR for forecasting and Seq2Seq for translation use recurrent models. One note: transformers now lead on many language tasks, but RNNs remain common, especially for time series.

Slide 8
Let's compare them side by side. CNNs work on grid like data, mainly images, and their key idea is filters that slide across the input. RNNs work on sequences, and their key idea is memory carried from step to step. CNN tasks include image classification and object detection. RNN tasks include translation, speech recognition, and forecasting. A simple rule: if the data is a picture, think CNN, and if the order of the data matters, think RNN.

Slide 9
Now regularization. Overfitting happens when a model learns the training data too well, including its noise, so it performs great on training data but poorly on new data. Regularization fights this by adding a penalty for model complexity, specifically for large weights, to the loss the model is trying to minimize. So the loss becomes the usual error plus lambda times a penalty. Lambda controls the strength. A simpler model, with smaller weights, tends to generalize better to unseen data.

Slide 10
There are two main kinds. L1 regularization, also called Lasso, adds the sum of the absolute values of the weights as the penalty. Its special property is that it pushes some weights all the way to exactly zero, which effectively removes those features. That makes it a form of automatic feature selection and gives you sparse models, handy when you have many irrelevant features. L2 regularization, also called Ridge or weight decay, adds the sum of the squared weights. It shrinks all weights smoothly toward zero but rarely makes them exactly zero, so every feature keeps some influence. It works well when most features matter. And remember lambda: if it's too high, the model underfits.

Slide 11
In practice, you'll meet these in many places. SageMaker's Linear Learner has hyperparameters for both L1 and L2. XGBoost has alpha for L1 and lambda for L2. For neural networks, dropout is another popular regularizer, which randomly switches off neurons during training so the network can't rely on any single one. And early stopping halts training when the validation error stops improving, which also prevents overfitting. You can tune these values with automatic model tuning.

Slide 12
So to recap, CNNs use filters to understand images, and RNNs carry memory to understand sequences. Regularization keeps models from overfitting by penalizing large weights, with L1 pushing some weights to zero and L2 shrinking them all smoothly. Match the architecture to your data, and add regularization to help your model generalize. Thanks for watching, and I'll see you in the next one.